

FULL-STACK ENGINEERING PROJECT

NovaBank

Bank Management System

Spring Boot 3.5 :: React 19 :: PostgreSQL 16 :: Docker

Complete Technical Documentation

How the system is designed, why each decision was made,
and how every moving part actually works.

27

tests passing

13

REST resources

6

database tables

2

user roles

Gurudutta Pradhan

guruduttapradhan140@gmail.com

Version 1.0 :: 37 pages :: August 2026

Demo environment - no real customer data or funds are involved.

Contents

1	Executive Summary	What the system is and the problem it solves
2	Feature Walkthrough	Every screen and what it does
3	Technology Stack	Each choice and the reason behind it
4	System Architecture	Layers, request flow, and package structure
5	Database Design	Schema, relationships, indexes, constraints
6	Security Architecture	JWT, BCrypt, roles, and the threats addressed
7	The Transfer Engine	Atomicity, locking, idempotency, retry
8	Backend Deep Dive	Layer by layer through the Spring Boot code
9	Frontend Deep Dive	React structure, state, design system, charts
10	API Reference	Endpoints, payloads, and the error contract
11	Testing Strategy	What is tested, how, and the results
12	Deployment	Docker, Compose, and configuration
13	Limitations and Roadmap	An honest account of what is not built
14	Demo Script	How to present this in an interview

How to read this document

Chapters 1-5 describe what was built. Chapters 6-9 explain how and, more importantly, why - these are the sections an interviewer is most likely to probe. Chapters 10-14 are reference material and a suggested demo run-through.

CHAPTER 1

Executive Summary

What the system is and the problem it solves

1

1.1 What NovaBank is

NovaBank is a working online-banking platform. A customer can register, open accounts of different types, deposit and withdraw cash, transfer money to other customers, save payees, review and filter their transaction history, and download a statement as CSV. An administrator gets a separate back office: bank-wide metrics, customer and account search, the ability to freeze or close an account or disable a login, and a read-only view of an immutable audit trail.

It is a full-stack system: a React 19 single-page application talking to a Spring Boot 3.5 REST API, backed by PostgreSQL, with the whole stack runnable through Docker Compose in one command.

1.2 Why a banking domain

Most portfolio projects are CRUD - create a row, read it back, edit it, delete it. Banking is interesting precisely because CRUD is not enough. Moving money raises problems that a to-do list never does:

- A transfer touches two rows and must be all-or-nothing. A crash halfway through must not destroy money.
- Two requests can hit the same balance at the same instant. Both may read 'you have 1000' and both may then approve a withdrawal of 700.
- Networks drop. A client that retries a transfer must not be charged twice.
- Fractions of currency cannot be stored in a binary floating-point type without drifting.
- Every action needs to be attributable afterwards, including the ones that failed.

Each of those has a specific, defensible engineering answer in this codebase, and those answers are the substance of this document.

1.3 The five decisions that define the system

Double-entry ledger

A transfer writes two immutable rows - a debit on the sender and a credit on the receiver - sharing one reference. Balances can always be reconciled against the ledger.

Pessimistic locking with a fixed lock order

Balances are read with `SELECT ... FOR UPDATE`, and when two accounts are involved they are locked in ascending account-number order. That removes both lost updates and deadlocks.

Idempotency keys backed by a unique index

A client-supplied key makes a retried transfer return the original result instead of moving the money again. The database constraint, not the application check, is the real guarantee.

Retry outside the transaction boundary

Lock contention still produces transient failures. A separate bean owns a bounded retry with jittered backoff, because a retry has to begin a brand-new transaction to be of any use.

Short access tokens, revocable refresh tokens

Access tokens live 15 minutes. Refresh tokens are stored server-side as SHA-256 hashes and rotated on every use, which restores the revocation that plain JWTs give up.

The honest framing

This is a demonstration system, not a production bank. It has no real KYC, no regulatory reporting, no settlement with external banks, and no HSM-backed key management. What it does have is a correct core: money is conserved, concurrent access is safe, and every action is recorded. Chapter 13 sets out the gaps explicitly.

CHAPTER 2

Feature Walkthrough

Every screen and what it does

2

2.1 Customer experience

Screen	What it does
Register	Name, email, 10-digit phone, password, optional date of birth and address. Password strength is shown live and enforced server-side. On success the user is signed in immediately and lands on the Accounts screen.
Sign in	Email and password, exchanged for an access + refresh token pair. Two one-click demo logins are offered so a reviewer never has to type credentials.
Dashboard	Total balance across all accounts, net cash flow this month, money in and out, a six-month cash-flow area chart, spending broken down by category, an account list, and the eight most recent transactions.
Accounts	Cards for each account showing balance, type, status, interest rate, daily limit and open date. Deposit, withdraw and statement download from the card. Opening a new account picks a type and funds it.
Transfer	Choose a source account, enter or pick a destination, set an amount and category. Saved payees fill the destination in one click. Success shows a receipt with the transaction reference and the new balance.
Transactions	Search by description or reference, filter by account, type and date range, page through results, export the range as CSV.
Payees	Add a payee by account number - the server verifies it exists and returns the real holder name before saving. Mark favourites; they sort first on the transfer screen.
Profile	Update name, phone and address. Change password, which revokes every active session on every device.

2.2 Administrator experience

Screen	What it does
Overview	Total customer holdings, account counts by state, 30-day transaction volume, new customers this month, and a daily credit-versus-debit bar chart.
Customers	Search by name, email or phone; filter by role. Enable or disable a login - disabling also revokes every refresh token, so the user is signed out everywhere immediately. Toggle the KYC flag.
Accounts	Search every account in the bank. Freeze (debits blocked, credits still accepted), unfreeze, or close. Closing requires a zero balance.
Audit trail	Every security- and money-relevant action, including failures: rejected logins, insufficient-funds attempts, breached daily limits, and attempts to touch another customer's account. Searchable and filterable by action.

2.3 Account products and their rules

The three account types are not cosmetic - each changes what the transfer engine will allow.

Type	Interest	Min. opening	Spendable balance	Notes
Savings	3.50% p.a.	Rs. 500	balance minus Rs. 500 floor	Cannot be drawn below the minimum balance.
Current	None	Rs. 0	balance plus Rs. 10,000 overdraft	May go negative into the arranged buffer.
Fixed Deposit	7.10% p.a.	Rs. 5,000	zero	Locked; debits are refused until maturity.

Account status is a second, independent gate. An ACTIVE account accepts money in and out. A FROZEN account still accepts credits but refuses debits - which is the behaviour a real bank wants, because freezing an account should not bounce an incoming salary. A CLOSED account is terminal and requires a zero balance to enter.

A small design decision that shows domain thinking

Splitting 'can this account be credited?' from 'can it be debited?' into two separate predicates - `isCreditable()` and `isDebitable()` - is what makes the frozen-account behaviour fall out naturally instead of needing a special case at every call site.

CHAPTER 3

Technology Stack

Each choice and the reason behind it

3

Every entry below is a decision, not a default. An interviewer will ask 'why this and not that' about at least one of them.

3.1 Backend

Technology	Version	Why this one
Java	21 (LTS)	Records make DTOs one line with correct equals/hashCode. Pattern matching for switch. Virtual threads for cheap request concurrency. LTS means long support.
Spring Boot	3.5.6	Auto-configuration removes hundreds of lines of boilerplate; starters give a coherent, version-aligned dependency set; it is the de facto standard for JVM services.
Spring Security	6.5	Battle-tested authentication and authorization. Writing this yourself is how security holes are created.
Spring Data JPA	3.5	Derived queries remove most DAO code, while @Query and @Lock give full control where it matters.
Hibernate	6.6	JPA provider. Supplies the pessimistic locking, optimistic versioning and dirty checking the transfer engine relies on.
Flyway	11	Versioned, reviewable, repeatable schema migrations. The schema becomes part of the codebase and its history.
PostgreSQL	16	Real MVCC, proper SELECT ... FOR UPDATE semantics, NUMERIC for exact decimals, mature and free.
H2	2.3	In-memory database for local runs and tests. The same portable migration runs on both engines, so dev and prod share one schema definition.
JWT	0.12.6	Focused, modern JWT library. The 0.12 API makes it hard to accidentally accept an unsigned token.
springdoc-openapi	2.8	Generates OpenAPI 3 and Swagger UI from the controllers, so the docs cannot drift from the code.

3.2 Frontend

Technology	Version	Why this one
React	19	Component model, huge ecosystem, the framework most teams actually use.
Vite	6	Sub-second dev server start and hot reload; Rollup-based production builds with automatic code splitting.
React Router	7	Nested routes let the authenticated shell wrap every private page once instead of repeating it.
Tailwind CSS	4	Utility classes keep styling next to the markup. v4's CSS-variable theming makes light/dark a single token swap rather than duplicated class lists.
Recharts	3	Composable SVG charts built on D3 scales, with the React API rather than imperative D3 code.
Axios	1.7	Interceptors are the reason: one place to attach the bearer token and one place to handle silent token refresh.
lucide-react	latest	Consistent, lightweight, tree-shakeable icon set.
react-hot-toast	2	Small, accessible toast notifications.

Why no Redux, and why no TypeScript?

Redux would be overhead here: the only genuinely global state is the authenticated user and the theme, both of which Context handles in a few dozen lines. Server data is fetched per screen and does not need a client-side store. TypeScript would be a real improvement and is listed in the roadmap - the honest reason it is absent is scope, and saying so is better than inventing a justification.

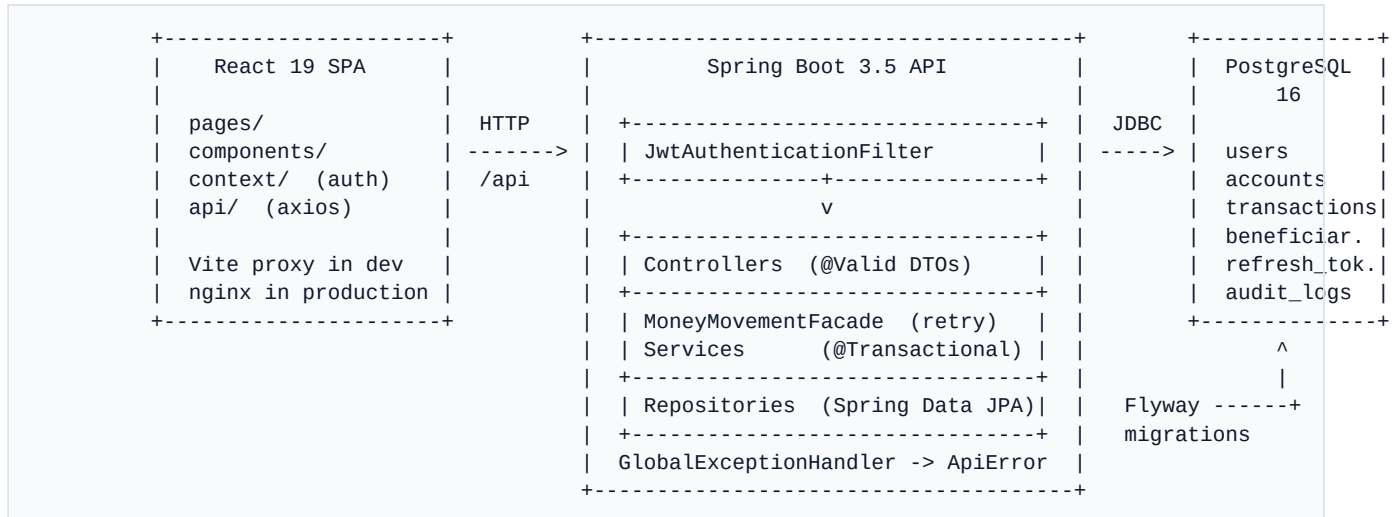
CHAPTER 4

System Architecture

Layers, request flow, and package structure

4

4.1 The big picture



Three tiers. The browser never talks to the database, and the API never renders HTML.

4.2 Why a layered architecture

Each layer has exactly one job, and each depends only on the layer below it. That is not dogma - it is what makes the code testable. Because services depend on repository interfaces rather than on a database, the entire business-rule test suite runs with mocks in about two seconds.

Layer	Responsibility	Explicitly NOT its job
Controller	Map HTTP to method calls, validate the request body with <code>@Valid</code> , choose the status code.	Business rules, database access, error formatting.
Facade	Own the retry policy for money movement, outside the transaction boundary.	Business rules - it simply calls the service again.
Service	Business rules, transaction boundaries, authorization checks, audit writes.	HTTP concerns. A service never sees a request or a response.
Repository	Data access, queries, locking hints.	Any decision about what should happen.
Entity	The persistent model plus small invariants such as <code>credit()</code> and <code>debit()</code> .	Ever being serialised to a client - DTOs exist for that.

4.3 Package by feature, not by layer

The Java packages are organised by domain concept - account, transaction, auth, admin - and each contains its own entity, repository, service, controller and DTOs. The common alternative, packaging by technical layer (all controllers together, all services together), scatters one feature across five directories and means every change touches five places.

```

com.gurudutta.bank
|
+-- BankApplication.java          entry point; enables auditing + scheduling
|
+-- account/                     Account, AccountType, AccountStatus,
|   +-- dto/                     AccountRepository, AccountService, AccountController
|
+-- transaction/                 the ledger and all money movement
|   +-- Transaction.java         one immutable ledger line
|   +-- TransactionService.java  deposit / withdraw / transfer - one attempt each
|   +-- MoneyMovementFacade.java retry policy (deliberately a separate bean)
|   +-- TransactionQueryService.java read side: search, dashboard, CSV statement
|   +-- InterestScheduler.java  monthly interest job
|
+-- auth/                       register, login, refresh, logout + RefreshToken
+-- user/                       User entity, profile and password management
+-- beneficiary/               saved payees
+-- admin/                     back-office operations
+-- audit/                     append-only AuditLog and AuditService
|
+-- security/                   JwtService, JwtAuthenticationFilter, principal
+-- config/                    SecurityConfig, OpenApiConfig, auditing, demo seed
+-- common/                    BaseEntity, exceptions, ApiError, PageResponse, Money

```

One feature, one folder.

4.4 The read/write split inside the transaction package

TransactionService handles writes; TransactionQueryService handles reads. They are separated because their requirements genuinely differ. Writes need row locks, transaction boundaries and audit entries. Reads need none of that and run under `@Transactional(readOnly = true)`, which lets Hibernate skip dirty-check bookkeeping and would allow routing to a read replica if one were ever added. It is the same instinct behind CQRS, applied at the scale this project warrants rather than as a full architectural pattern.

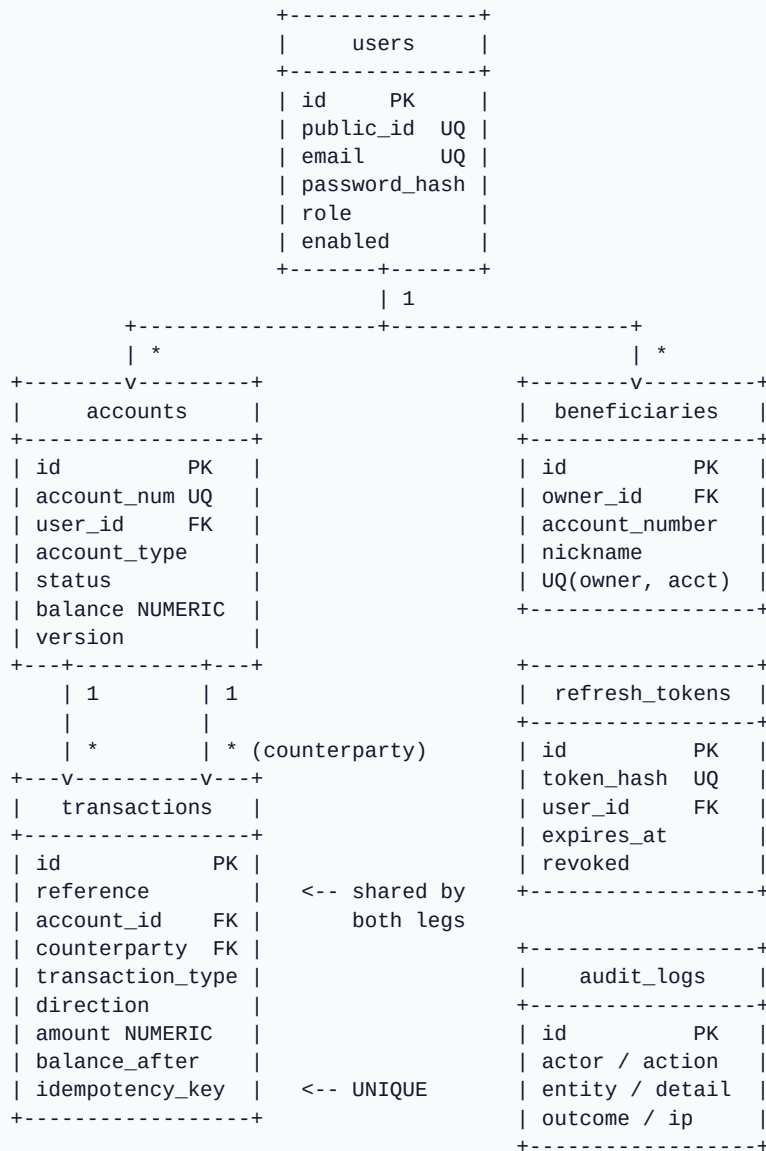
CHAPTER 5

Database Design

Schema, relationships, indexes, constraints

5

5.1 Entity relationships



Six tables. `audit_logs` is intentionally unlinked by foreign key - see 5.5.

5.2 Money is NUMERIC, never FLOAT

Every monetary column is NUMERIC(19,4) and every Java field is BigDecimal. This is the single most important schema decision. Binary floating point cannot represent 0.10 exactly; the nearest double is slightly more than a tenth. Add it ten times and the result is not 1.0.

```
jshell> 0.1 + 0.2
$1 ==> 0.30000000000000004

jshell> double total = 0; for (int i = 0; i < 10; i++) total += 0.1; total
$2 ==> 0.9999999999999999

// With BigDecimal, the arithmetic is exact and the scale is explicit:
new BigDecimal("0.10").add(new BigDecimal("0.20")) // 0.30
```

In a ledger, the difference between 0.9999999999999999 and 1.00 is money that vanished.

Scale 4 rather than 2 is also deliberate. Settlement happens at two decimal places, but intermediate values - accrued interest, a monthly rate derived from an annual one - need more precision before they are rounded for posting.

5.3 Table reference

users

Column	Type	Notes
id	BIGINT identity	Primary key. Never leaves the server.
public_id	VARCHAR(36) UNIQUE	UUID exposed to clients, so sequential ids cannot be enumerated to count customers.
email	VARCHAR(160) UNIQUE	Login identifier, stored lower-case.
password_hash	VARCHAR(100)	BCrypt digest. The plain password is never stored or logged.
role	VARCHAR(20) CHECK	CUSTOMER or ADMIN, enforced by a CHECK constraint.
enabled	BOOLEAN	Disabling blocks sign-in and revokes live sessions.
kyc_verified	BOOLEAN	Set by an administrator.

accounts

Column	Type	Notes
account_number	VARCHAR(20) UNIQUE	12 digits, generated with SecureRandom and collision-checked, so numbers are not guessable from one another.
user_id	BIGINT FK	Owner.
account_type	VARCHAR(20) CHECK	SAVINGS / CURRENT / FIXED_DEPOSIT.
status	VARCHAR(20) CHECK	ACTIVE / FROZEN / CLOSED.
balance	NUMERIC(19,4)	Exact decimal.
daily_transfer_limit	NUMERIC(19,4)	Checked against the sum of today's debits.
version	BIGINT	Hibernate @Version. Optimistic-lock guard behind the pessimistic one.

transactions - the ledger

Column	Type	Notes
reference	VARCHAR(36)	Shared by both legs of a transfer. This is what makes the pair a double entry rather than two unrelated rows.
account_id	BIGINT FK	The account this line belongs to.
counterparty_account_id	BIGINT FK NULL	The other side. Null for cash deposits and withdrawals.
direction	VARCHAR(10) CHECK	CREDIT or DEBIT.
amount	NUMERIC(19,4) CHECK > 0	Always positive; direction carries the sign.
balance_after	NUMERIC(19,4)	Running balance, denormalised so a statement does not re-sum the whole history for every row.
idempotency_key	VARCHAR(80) UNIQUE	Nullable, unique when present. This index is the real de-duplication guarantee.

5.4 Indexes and why each exists

Index	Supports	Without it
idx_tx_account_created (account_id, created_at DESC)	The statement query: filter by account, order by date descending.	A full scan plus an in-memory sort on every page of history.
uk_tx_idempotency (idempotency_key)	Enforcing single application of a retried request.	Two concurrent retries could both pass the service check and both post.
idx_accounts_number (account_number)	Every lookup, since the API addresses accounts by number, not id.	A scan on the hot path of every transfer.
uk_beneficiary_owner_account (owner_id, account_number)	Preventing the same payee being saved twice.	Duplicate rows in the payee list.
idx_audit_created / idx_audit_action	The admin audit screen's default ordering and its action filter.	Slow queries that get worse as the log grows - and it only ever grows.

5.5 Constraints as the last line of defence

The application validates input, but constraints in the schema are what hold when the application is wrong. CHECK (amount > 0) makes a negative ledger line impossible even if a future code path forgets to validate. The unique index on idempotency_key holds even when two requests race past the service-level lookup at the same microsecond. Foreign keys make an orphaned account unrepresentable.

audit_logs deliberately has no foreign key to users. An audit row must survive the deletion of whatever it describes - that is the point of an audit trail - so it stores the actor as text rather than as a reference that could cascade away.

5.6 Migrations, not auto-DDL

Flyway owns the schema and Hibernate is configured with `ddl-auto: validate`. The migration is a reviewable file in version control; Hibernate's only job at startup is to fail fast if an entity and the migrated schema have drifted apart.

Why ddl-auto: update is a trap

It looks convenient and it works in development. In production it silently issues ALTER statements nobody reviewed, it will never drop or rename anything (so mistakes accumulate forever), and there is no way to roll it back or to reproduce the same schema on another machine. 'validate' converts a class of silent production incidents into a loud startup failure on a developer's laptop.

CHAPTER 6

Security Architecture

JWT, BCrypt, roles, and the threats addressed

6

6.1 The filter chain

```

Request
|
v
[ CorsFilter ]                origin allowed?
|
v
[ JwtAuthenticationFilter ]    extends OncePerRequestFilter
|  read "Authorization: Bearer <jwt>"
|  verify HMAC-SHA256 signature + issuer + expiry
|  reject if the token type is "refresh" rather than "access"
|  load UserDetails, populate the SecurityContext
|  (on any failure: do nothing, stay anonymous - never throw)
v
[ AuthorizationFilter ]       route rules from SecurityConfig
|  /api/v1/auth/**            -> permitAll
|  /api/v1/admin/**           -> hasRole('ADMIN')
|  everything else             -> authenticated
v
[ Controller ] -> [ @PreAuthorize ] -> [ Service ] -> ownership check

```

Authentication answers 'who are you'; authorization answers 'may you'. They are separate steps.

6.2 Passwords

Passwords are hashed with BCrypt at strength 10. Two properties matter. BCrypt is deliberately slow, so brute-forcing a stolen hash is expensive rather than instant. And it generates a random salt per password and stores it inside the hash string, so two users who choose the same password get different hashes and a precomputed rainbow table is useless.

```

$2a$10$N9qo8uL0ickgx2ZMRZoMye IjZAgcfl7p92ldGxad68LJZdL17lhwy
| | | |
| | +-- 22-char salt          +-- 31-char hash
| +-- cost factor 10 -> 2^10 = 1024 key-expansion rounds
+-- algorithm version

```

Everything BCrypt needs to verify a password is inside the string - no separate salt column.

Strength is a tuning decision, not a fixed truth: each increment doubles the work. Ten is a reasonable balance for a demo; a production system should measure and pick the highest cost that keeps login under roughly 250 ms on real hardware.

6.3 The two-token design

	Access token	Refresh token
Lifetime	15 minutes	7 days
Sent	On every request	Only to /auth/refresh
Stored server-side	No - fully stateless	Yes, as a SHA-256 hash
Revocable	No	Yes - delete or flag the row
If stolen	Small window of abuse	Rotation exposes the theft on next use

A JWT is self-contained: the server can verify it without a database lookup, which is exactly what makes it scalable and exactly what makes it impossible to un-issue. The two-token split resolves that tension. The token sent constantly is disposable and short-lived; the token that grants long sessions is stored, and therefore revocable.

Refresh-token rotation

Each refresh consumes the old token and issues a new pair. If an attacker steals a refresh token and uses it, the legitimate user's copy is already revoked, so their next refresh fails and the compromise surfaces as a forced re-login rather than as silent, indefinite access.

Why only the hash is stored

For the same reason passwords are hashed: a database leak must not hand out working sessions. A plain SHA-256 with no salt is correct here, unlike for passwords - the input is a signed JWT with well over 200 bits of entropy, so there is nothing to brute-force or to look up in a rainbow table.

6.4 Threats and the specific control for each

Threat	Control in this codebase
SQL injection	All access goes through JPA with bound parameters. No string concatenation builds a query anywhere in the code.
Password theft from a DB dump	BCrypt, per-password salt, cost factor 10.
Broken object-level authorization (IDOR)	requireOwnedAccount() checks ownership on every account access, and returns 404 rather than 403 so the response is not an existence oracle.
Privilege escalation via the API	RegisterRequest has no role field, so it cannot be set from outside. Role changes are admin-only.
Mass assignment	Controllers bind to purpose-built DTOs, never to entities.
Sensitive data exposure	Responses are DTOs. password_hash and internal ids are never serialised.
User enumeration	Login failures return one identical message; hideUserNotFoundExceptions keeps the timing similar too.
Session fixation / stale sessions	Stateless API, no HttpSession. Password change and admin disable both revoke every refresh token.
CSRF	Disabled deliberately: authentication is a header, not a cookie, and a cross-site form cannot set an Authorization header.
Cross-origin abuse	CORS allow-list is configuration, not a wildcard.
Stack-trace leakage	GlobalExceptionHandler returns a generic message; the detail is logged server-side only. server.error.include-stacktrace: never.
Clickjacking / MIME sniffing	nginx sends X-Frame-Options: DENY and X-Content-Type-Options: nosniff.

Where the tokens are stored on the client, and the trade-off

Tokens live in localStorage. That is convenient and survives a refresh, but it is readable by any JavaScript running on the page, so a successful XSS attack can steal the session. The more secure option is an httpOnly, Secure, SameSite=Strict cookie, which JavaScript cannot read - at the cost of needing CSRF protection back. Naming this trade-off honestly is worth more in an interview than pretending localStorage is fine.

CHAPTER 7

The Transfer Engine

Atomicity, locking, idempotency, retry

7

This is the chapter to know cold. Everything else in the project is competent CRUD; this is the part that is actually difficult, and it is where a good interviewer will spend their time.

7.1 The problem: a lost update

Two withdrawals of 700 arrive at the same instant against a balance of 1000. Without coordination, both read 1000, both conclude the withdrawal is affordable, and both write their result. One overwrites the other.

TIME	Transaction A	Transaction B	BALANCE
----	-----	-----	-----
t1	read balance -> 1000		1000
t2		read balance -> 1000	1000
t3	1000 >= 700, approve		1000
t4		1000 >= 700, approve	1000
t5	write 1000 - 700 = 300		300
t6		write 1000 - 700 = 300	300
			^
			1400 withdrawn from an account holding 1000.
			The bank has silently created 700 out of nothing.

The classic lost update. Both transactions were individually correct.

7.2 The fix: a pessimistic row lock

The balance is read with `SELECT ... FOR UPDATE`, which takes a write lock on the row. The second transaction blocks at t2 until the first commits, then re-reads the balance the first one actually wrote.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Account a WHERE a.accountNumber = :accountNumber")
Optional<Account> findByAccountNumberForUpdate(@Param("accountNumber") String accountNumber);
```

AccountRepository - the single most important five lines in the backend.

TIME	Transaction A	Transaction B	BALANCE
----	-----	-----	-----
t1	SELECT ... FOR UPDATE -> 1000		1000
t2		SELECT ... FOR UPDATE	
		... BLOCKED, waiting	1000
t3	1000 >= 700, approve	(waiting)	1000
t4	write 300, COMMIT	(waiting)	300
t5		unblocks, re-reads -> 300	300
t6		300 >= 700 ? NO -> reject	300

Serialized. The second withdrawal is correctly refused.

7.3 Pessimistic or optimistic?

Both are implemented. Optimistic locking (a `@Version` column) lets both transactions proceed and fails the second at commit time; it is ideal when conflicts are rare, because it takes no locks at all. Pessimistic locking blocks up front; it is right when conflicts are likely and when a late failure is expensive.

Money movement uses pessimistic locking as the primary control, because the hot account of an active customer genuinely does see concurrent writes, and because telling a user their transfer failed after they pressed Send is a much worse outcome than making them wait 5 ms. The @Version column remains as a second guard: it catches any future write path that forgets to take the lock.

7.4 Deadlock, and the ordering that prevents it

A lock introduces a new failure mode. Consider Priya transferring to Rahul at the same moment Rahul transfers to Priya, with each transaction locking its own source account first:

Transaction A (Priya -> Rahul) ----- lock Priya [ok] lock Rahul ... waits for B	Transaction B (Rahul -> Priya) ----- lock Rahul [ok] lock Priya ... waits for A
--	--

Neither can proceed. Neither will ever release.

A circular wait. Both transactions hold what the other needs.

The fix is to remove the circularity by imposing a total order on lock acquisition. Both accounts are sorted by account number and locked lowest-first, regardless of which is the sender:

```
// Deadlock avoidance: always acquire in the same global order, so a cycle
// is structurally impossible rather than merely unlikely.
String first = min(request.fromAccountNumber(), request.toAccountNumber());
String second = max(request.fromAccountNumber(), request.toAccountNumber());

Account firstLocked = lockAccount(first);    // SELECT ... FOR UPDATE
Account secondLocked = lockAccount(second);  // SELECT ... FOR UPDATE

// Work out which of the two is actually the source.
Account source = firstLocked.getAccountNumber().equals(request.fromAccountNumber())
    ? firstLocked : secondLocked;
Account destination = source == firstLocked ? secondLocked : firstLocked;
```

TransactionService.transfer(). Now both transactions want Priya's row first; one wins, the other waits, and both complete.

Verified, not assumed

TransferConcurrencyIT starts 20 transfers in both directions simultaneously behind a CountdownLatch and asserts both balances return exactly to their starting values. The test would hang and time out if the ordering were wrong.

7.5 Retry: what locking alone does not solve

During development, firing twelve concurrent transfers produced HTTP 500s. The cause was genuine and worth understanding: under contention a database may refuse a transaction rather than queue it - a lock-wait timeout, or being selected as a deadlock victim. H2 raises SQLSTATE 40001 in this situation. These are transient failures. Nothing was applied, and the correct response is to try again, not to tell the customer their transfer failed.

The subtle part is where the retry can live. Once a transaction is marked rollback-only, every further statement on it fails - so a retry loop inside the service method would simply fail four more times inside the same doomed transaction. The retry has to begin a brand-new transaction, which means it must sit outside the transactional boundary. Because Spring's @Transactional works through a proxy, 'outside the boundary' means 'in a different bean':

```

@Service
public class MoneyMovementFacade {           // NOT @Transactional

    private final TransactionService transactionService;

    public TransferResponse transfer(TransferRequest request) {
        return withRetry("transfer", () -> transactionService.transfer(request));
    }
    // ^ crosses the proxy -> new transaction

    private <T> T withRetry(String operation, Supplier<T> action) {
        for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
            try {
                return action.get();
            } catch (ConcurrencyFailureException ex) {
                // Covers optimistic, pessimistic and deadlock-victim failures alike -
                // they share this superclass because the remedy is the same for all three.
                if (attempt == MAX_ATTEMPTS) break;
                backoff(attempt);           // exponential, with jitter
            }
        }
        throw new BusinessException("The account is busy with another transaction. "
            + "Please try again in a moment.", HttpStatus.CONFLICT, "CONCURRENT_MODIFICATION");
    }
}

```

Business failures - insufficient funds, frozen account - are BusinessExceptiones and are never retried.

The jitter matters. Without randomness, every loser of a collision would wake at the same instant and collide again, turning one contended moment into a repeating one.

Before and after

Twelve concurrent transfers, measured against the running API. Before: 3 of 12 applied, 9 returned HTTP 500. After: 12 of 12 applied, sender and receiver deltas matched exactly. In both runs no money was created or destroyed - the locking was already correct - but only the second run is an acceptable user experience.

7.6 Idempotency

A customer presses Send twice. Or their phone loses signal after the request is sent but before the response arrives, and the app retries. Without protection, the money moves twice.

The client generates a UUID per transfer attempt and sends it as idempotencyKey. The server checks it before doing any work and, if the key has been seen, returns the original result. Crucially, the column also carries a UNIQUE index - because two concurrent retries could both pass the application-level check in the same microsecond. The application check is the fast path; the database constraint is the guarantee.

```

// Client: one key per form session, minted when the form resets - so a
// double-click reuses it, while a genuinely new transfer gets a fresh one.
const [idempotencyKey, setIdempotencyKey] = useState(newIdempotencyKey)

// Server: check before doing anything at all.
Transaction replayed = findByIdempotencyKey(request.idempotencyKey());
if (replayed != null) {
    return toTransferResponse(replayed);    // original result, no money moved
}

```

Verified end-to-end: three identical submissions debit the account exactly once.

7.7 Double-entry bookkeeping

A transfer writes two rows, not one - a DEBIT on the sender and a CREDIT on the receiver - sharing a single reference UUID. This is a 500-year-old accounting principle and it earns its place: every rupee that leaves one account is recorded arriving in another, so the ledger can always be proved to balance. Summing every row for an account must reproduce its balance; if it does not, something is wrong and can be found.

Ledger rows are never updated or deleted. A mistake is corrected by posting a compensating entry, exactly as a real bank does, which keeps the history intact and auditable.

7.8 The complete transfer path

```
POST /api/v1/transactions/transfer
|
+-- 1. JwtAuthenticationFilter      verify signature, populate SecurityContext
+-- 2. SecurityFilterChain          route requires authentication
+-- 3. @Valid TransferRequest       amount > 0, <= 2 dp, accounts non-blank
+-- 4. MoneyMovementFacade          enter retry loop (attempt 1 of 4)
|
+-- 5. TransactionService.transfer() @Transactional STARTS HERE
|
|   +-- validate amount             positive, within the single-transaction cap
|   +-- reject self-transfer        before taking any lock
|   +-- idempotency lookup          seen this key? replay and return
|   +-- lock LOWER account number   SELECT ... FOR UPDATE
|   +-- lock HIGHER account number  SELECT ... FOR UPDATE <-- consistent order
|   +-- authorise ownership         caller owns the source, or is an admin
|   +-- source debitible?           status ACTIVE, spendable balance sufficient
|   +-- destination creditable?    status ACTIVE or FROZEN
|   +-- daily limit                 sum of today's debits + this amount <= limit
|   +-- source.debit(amount)         in memory
|   +-- destination.credit(amount)  in memory
|   +-- write DEBIT leg             shared reference, balance_after, idempotency key
|   +-- write CREDIT leg            same reference
|   +-- audit                       REQUIRES_NEW - survives a rollback
|   |
|   +-- COMMIT                      both balances and both legs, atomically
|
+-- 6. 201 Created + TransferResponse reference, amount, new source balance
```

Any failure before COMMIT rolls back every change. There is no state in which one leg exists without the other.

CHAPTER 8

Backend Deep Dive

Layer by layer through the Spring Boot code

8

8.1 Transaction management and propagation

@Transactional wraps a method in a database transaction: it commits on normal return and rolls back on an unchecked exception. It works by proxying the bean, which has two consequences worth knowing, because both are common interview questions and both appear in this codebase.

Self-invocation does not work

Calling a @Transactional method from another method of the same class goes through 'this', not through the proxy, so the annotation is silently ignored. The monthly interest job originally had this bug: a scheduled method looping over accounts and calling a @Transactional method on itself would have run the whole batch in no transaction at all. The fix was to move the scheduler into its own bean, InterestScheduler, so the call crosses a proxy boundary and each account is genuinely committed independently.

REQUIRES_NEW for auditing

AuditService.record() is annotated @Transactional(propagation = REQUIRES_NEW). It suspends the caller's transaction and runs in its own. That is precisely what an audit trail needs: if a transfer is rejected for insufficient funds and its transaction rolls back, the record that the attempt was made must still survive. A failed attempt is exactly what a compliance officer wants to see.

Propagation	Behaviour	Used here for
REQUIRED (default)	Join the caller's transaction, or start one.	All normal service methods.
REQUIRES_NEW	Suspend the caller's transaction, run in a fresh one.	Audit logging, so records survive a rollback.
SUPPORTS	Join if one exists, otherwise run without.	Not used.
MANDATORY	Throw if no transaction exists.	Not used.
NEVER / NOT_SUPPORTED	Refuse / suspend.	Not used.

The cost of REQUIRES_NEW, and the mitigation

A suspended transaction still holds its connection while the inner one takes a second. A money-movement request therefore occupies two pool connections at its peak. With a small pool and enough concurrency, requests holding row locks could end up queuing for a connection that only a lock-holder can release - a self-inflicted deadlock. The pool is sized at 24 with leak detection enabled for exactly this reason.

8.2 Avoiding the N+1 query problem

Lazy associations are the default here, which is correct - loading a transaction should not drag in its account, its account's owner and everything else. But rendering a 25-row statement then triggers 25 extra `SELECT`s, one per row, to resolve each account. That is the N+1 problem: 1 query for the list and N for the details.

```
@Query(value = ""
    SELECT t FROM Transaction t
    JOIN FETCH t.account a
    JOIN FETCH a.owner
    LEFT JOIN FETCH t.counterpartyAccount
    WHERE a.id IN :accountIds
    AND (:type IS NULL OR t.type = :type)
    ...
    """,
    countQuery = "" SELECT COUNT(t) FROM Transaction t WHERE ... """)
Page<Transaction> search(...);
```

JOIN FETCH loads the associations in the same query. A separate countQuery is required because a JOIN FETCH cannot be counted.

The same instinct applies to the dashboard's six-month chart. The obvious implementation loops over six months and queries once per month - the N+1 problem again, just with months instead of rows. Instead, the whole window is fetched in one query and bucketed in Java.

8.3 Validation in three layers

Layer	Example	Catches
Bean Validation on the DTO	@DecimalMin("0.01"), @Pattern for the password rules	Malformed input, before any business code runs. Returns 400 with a per-field breakdown.
Business rules in the service	spendable balance, account status, daily limit	Requests that are well-formed but not permitted by the domain. Returns 422.
Database constraints	CHECK (amount > 0), UNIQUE (idempotency_key)	Anything the first two layers missed, including genuine races. Returns 409.

The client also validates, but that is purely a convenience so the user gets feedback without a round trip. It is not a control - anyone can edit JavaScript in their own browser, or skip the browser entirely and call the API with curl.

8.4 One error contract

@RestControllerAdvice centralises error handling so no controller contains a try/catch and every failure - validation, domain, security, database, unexpected - comes back in the same JSON shape. The machine-readable code field lets the React client branch on the failure type without parsing English.

Exception	Status	code
ResourceNotFoundException	404	RESOURCE_NOT_FOUND
DuplicateResourceException	409	DUPLICATE_RESOURCE
InsufficientFundsException	422	INSUFFICIENT_FUNDS
InvalidOperationException	422	e.g. DAILY_LIMIT_EXCEEDED, ACCOUNT_NOT_DEBITABLE
MethodArgumentNotValidException	400	VALIDATION_FAILED (+ per-field violations)

Exception	Status	code
BadCredentialsException	401	BAD_CREDENTIALS
AccessDeniedException	403	ACCESS_DENIED
ConcurrencyFailureException	409	CONCURRENT_MODIFICATION
DataIntegrityViolationException	409	DATA_INTEGRITY_VIOLATION
NoResourceFoundException	404	ENDPOINT_NOT_FOUND
Exception (catch-all)	500	INTERNAL_ERROR - detail logged, never returned

A bug the tests caught

An unknown URL originally returned 500. Spring 6.1 raises NoResourceFoundException for an unmatched path, not the NoHandlerFoundException that was being handled, so a simple typo in a URL fell through to the catch-all. A 500 tells an on-call engineer the server is broken; the correct answer is 404. Found by SecurityAndApiIT, fixed by handling both types.

8.5 DTOs, and why entities never cross the wire

Every request and response is a Java record, separate from the JPA entity. Four reasons, in order of importance:

1. Security. Serialising the User entity would expose password_hash and the internal numeric id.
2. Mass assignment. Binding a request directly to an entity would let a caller set fields such as role or balance simply by including them in the JSON.
3. Lazy loading. Serialising an entity outside a transaction triggers LazyInitializationException on every lazy association.
4. Coupling. Without DTOs, a database column rename becomes a breaking API change.

Records are ideal for this: immutable by construction, with equals, hashCode and toString generated, in a single line.

8.6 Scheduled work

InterestScheduler runs at 01:00 on the first of each month and credits interest to every active savings account. Each account is committed in its own transaction, so one bad row cannot roll back the entire run, and each failure is logged and stepped over rather than aborting the batch.

CHAPTER 9

Frontend Deep Dive

React structure, state, design system, charts

9

9.1 Structure

```

frontend/src/
+-- main.jsx           root render; providers and the toast host
+-- App.jsx            route table and the RequireAuth guard
+-- index.css          design tokens + Tailwind v4 layer definitions
|
+-- api/
|   +-- client.js      axios instance, auth header, silent token refresh
|   +-- endpoints.js   every server call the app can make, in one place
|
+-- context/
|   +-- AuthContext.jsx session state, sign in / out, hydration
|   +-- ThemeContext.jsx light/dark, persisted, OS-preference aware
|
+-- hooks/
|   +-- useApi.js      loading / error / data + refetch; useDebounce
|
+-- components/
|   +-- ui/index.jsx    Button, Input, Select, Card, Modal, Badge, EmptyState...
|   +-- layout/         AppShell (sidebar + topbar), AuthLayout
|   +-- charts/         validated colour theme + the three chart components
|   +-- TransactionRow.jsx one statement line, used on three screens
|
+-- pages/
|   +-- admin/          Login, Register, Dashboard, Accounts, Transfer,
|                       Transactions, Beneficiaries, Profile, NotFound
|
+-- utils/format.js     currency, dates, masking, idempotency keys

```

Layout mirrors the backend's instinct: grouped by what a thing is for.

9.2 Silent token refresh

Access tokens expire after 15 minutes, so a user reading a page will hit a 401 mid-session. Bouncing them to the login screen would be a poor experience. An axios response interceptor catches the first 401, refreshes the token pair, and replays the original request.

```

let refreshPromise = null      // <-- the important line

api.interceptors.response.use(null, async (error) => {
  const { config, response } = error

  if (response?.status === 401 && !config._retried && !isAuthCall(config)) {
    config._retried = true      // only ever retry once

    // If six requests 401 at the same moment, all six await the SAME refresh.
    refreshPromise = refreshPromise || axios
      .post('/api/v1/auth/refresh', { refreshToken: tokenStore.refresh() })
      .then(res => { tokenStore.save(res.data); return res.data.accessToken })
      .finally(() => { refreshPromise = null })

    const freshToken = await refreshPromise
    config.headers.Authorization = `Bearer ${freshToken}`
    return api(config)          // replay the original request
  }
  return Promise.reject(toApiError(response))
})

```

The shared promise is not an optimisation - it is a correctness fix.

Why the shared promise matters

The dashboard fires several requests in parallel. If all of them 401 at once and each starts its own refresh, the server - which rotates refresh tokens on every use - accepts the first and rejects the rest, and the user is thrown out mid-session. Collapsing them onto one in-flight promise is what makes rotation and parallel requests coexist.

9.3 Routing and guards

React Router 7 nested routes let the authenticated shell - sidebar, top bar, footer - wrap every private page exactly once, with each page rendering into an Outlet. RequireAuth blocks unauthenticated access and remembers the attempted path so login can return the user there. Admin routes carry an additional adminOnly check.

Client guards are UX, not security

Everything in the browser is editable by the user. A route guard stops a customer seeing a broken admin screen; it is Spring Security rejecting the request that stops them reading admin data. This is worth saying out loud in an interview - it is a common place for candidates to over-claim.

9.4 The design system

Colours are CSS custom properties, not hard-coded Tailwind classes. Light and dark are two values of the same variable rather than two parallel sets of markup, so a component written once themes itself for free and the theme toggle is a single class on the html element.

```

:root {
  --surface-page:    #f6f8fb;
  --surface-card:    #ffffff;
  --surface-border:  #e2e8f0;
  --text-primary:    #0f172a;
  --text-secondary:  #475569;
}

.dark {
  --surface-page:    #070b14;
  --surface-card:    #0e1424;
  --surface-border:  #1e2942;
  --text-primary:    #f1f5f9;
  --text-secondary:  #94a3b8;
}

```

Six tokens carry the entire theme. Components reference roles, never raw hex.

Details that separate a polished interface from a merely functional one:

- Tabular figures on every monetary value, so digits do not jitter as balances change.
- Skeleton loaders that match the final layout, so nothing shifts when data arrives.
- Focus-visible rings left intact - a keyboard user's only cue for where they are.
- prefers-reduced-motion honoured; all animation collapses to near zero.
- Account numbers masked to the last four digits in lists, as a card receipt would.
- Submit buttons disable while a request is in flight, which is what actually prevents a double-submit reaching the API.
- aria-invalid and aria-describedby on invalid fields, so a screen reader announces the error rather than only showing it in red.

9.5 Charts

Chart colours were validated, not chosen by eye. The two series hues were checked for lightness band, chroma floor, colour-blind separation under simulated protanopia and deuteranopia, and contrast against the actual chart surface - in both light and dark mode. They clear the colour-blind separation threshold with a wide margin and exceed 3:1 contrast on both surfaces.

Chart	Form	Why this form
Cash flow	Area, two series, one shared y-axis	Change over time. A second y-axis is never used - it lets two series appear to cross wherever the author likes, which makes it the most misleading chart type in common use.
Spending by category	Horizontal bars, single hue, direct labels	Comparing lengths against a shared baseline is far more accurate than comparing pie angles. One hue because the axis already names each category, so colour has no work to do.
Daily volume (admin)	Grouped bars, two series	Discrete daily totals with two comparable measures.

Identity is never carried by colour alone: every multi-series chart has a legend with text labels, the category bars are directly labelled with their values, and credit versus debit in transaction lists is marked with a plus or minus sign as well as a colour.

CHAPTER 10

API Reference

Endpoints, payloads, and the error contract

10

Interactive documentation is generated from the controllers and served at `/swagger-ui.html`, so it cannot drift from the implementation.

10.1 Endpoints

Method	Path	Auth	Purpose
POST	/api/v1/auth/register	-	Create a customer, return a token pair
POST	/api/v1/auth/login	-	Exchange credentials for tokens
POST	/api/v1/auth/refresh	-	Rotate the token pair
POST	/api/v1/auth/logout	JWT	Revoke a refresh token
GET	/api/v1/auth/me	JWT	Current profile
GET	/api/v1/accounts	JWT	List the caller's accounts
POST	/api/v1/accounts	JWT	Open an account
GET	/api/v1/accounts/{number}	JWT	Account detail
DELETE	/api/v1/accounts/{number}	JWT	Close (requires a zero balance)
GET	/api/v1/accounts/{number}/statement	JWT	CSV statement for a date range
GET	/api/v1/transactions	JWT	Paged, filtered history
GET	/api/v1/transactions/dashboard	JWT	Aggregates for the home screen
POST	/api/v1/transactions/deposit	JWT	Credit an account
POST	/api/v1/transactions/withdraw	JWT	Debit an account
POST	/api/v1/transactions/transfer	JWT	Atomic transfer
GET	/api/v1/beneficiaries	JWT	List saved payees
POST	/api/v1/beneficiaries	JWT	Save a payee (holder name verified)
PUT	/api/v1/beneficiaries/{id}	JWT	Rename or favourite
DELETE	/api/v1/beneficiaries/{id}	JWT	Remove a payee
GET/PUT	/api/v1/users/me	JWT	Read / update profile
POST	/api/v1/users/me/password	JWT	Change password, revoke all sessions
GET	/api/v1/admin/stats	ADMIN	Bank-wide metrics
GET	/api/v1/admin/users	ADMIN	Search users
PATCH	/api/v1/admin/users/{id}/status	ADMIN	Enable / disable a login
PATCH	/api/v1/admin/users/{id}/kyc	ADMIN	Set the KYC flag
GET	/api/v1/admin/accounts	ADMIN	Search accounts
PATCH	/api/v1/admin/accounts/{n}/status	ADMIN	Freeze / unfreeze / close
GET	/api/v1/admin/audit-logs	ADMIN	Read the audit trail

10.2 Worked example: a transfer

```

POST /api/v1/transactions/transfer
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "fromAccountNumber": "900100100101",
  "toAccountNumber": "900100100201",
  "amount": 2500.00,
  "description": "July rent",
  "category": "Rent",
  "idempotencyKey": "8f14e45f-ceed-467a-9b2c-1d0a2e4b9c77"
}

--- 201 Created -----

{
  "reference": "7f0c25ea-9904-42ab-8d56-89a9f1af1b1a",
  "fromAccountNumber": "900100100101",
  "toAccountNumber": "900100100201",
  "toAccountHolder": "Rahul Verma",
  "amount": 2500.00,
  "sourceBalanceAfter": 558737.00,
  "completedAt": "2026-08-08T07:18:24.851Z"
}

```

10.3 The error contract

Every failure, from every endpoint, returns this shape. The status code says what kind of problem it is; the code field says exactly which one, so the client can branch on it.

```

--- 422 Unprocessable Entity -----
{
  "timestamp": "2026-08-08T07:18:24Z",
  "status": 422,
  "error": "Unprocessable Entity",
  "code": "INSUFFICIENT_FUNDS",
  "message": "Insufficient funds in account 900100100101. Available: 9500.00, requested: 9600.00",
  "path": "/api/v1/transactions/withdraw",
  "violations": null
}

--- 400 Bad Request (bean validation) -----
{
  "status": 400,
  "code": "VALIDATION_FAILED",
  "message": "One or more fields are invalid.",
  "violations": [
    { "field": "password", "message": "Password must contain an uppercase letter, a lowercase letter and a digit" },
    { "field": "phone", "message": "Phone must be exactly 10 digits" }
  ]
}

```

CHAPTER 11

Testing Strategy

What is tested, how, and the results

11

27 tests, all passing. Run them with `./mvnw test` from the backend directory.

Suite	Tests	Style	Covers
TransactionServiceTest	9	Unit, mocked repositories	Savings minimum balance, current-account overdraft, frozen accounts, single-transaction cap, daily limit, self-transfer, cross-customer access, both ledger legs, idempotent replay.
TransferConcurrencyIT	3	Integration, real threads	20 simultaneous transfers conserve the total exactly; 20 opposing transfers complete without deadlock; a repeated idempotency key charges once.
SecurityAndApiIT	15	Integration, full HTTP via MockMvc	401/403/404 behaviour, refresh-token misuse, cross-customer reads and debits, role escalation, validation errors, CSV export, and the open-deposit-read happy path.

11.1 The test that matters most

A concurrency bug that only appears under real parallelism cannot be caught by a sequential test. `TransferConcurrencyIT` creates 20 threads, holds them all at a `CountDownLatch`, releases them at the same instant, and then asserts the invariants.

```
CountDownLatch startGate = new CountDownLatch(1);

for (int i = 0; i < 20; i++) {
    tasks.add(() -> {
        authenticateAs(aliceUser);           // SecurityContext is thread-local
        startGate.await();                   // hold every thread here...
        moneyMovement.transfer(...);         // ...then release them together
        return null;
    });
}
startGate.countDown();                      // maximum real contention

// The invariant that actually matters:
assertThat(aliceFinal.add(bobFinal))
    .as("total money in the system is unchanged")
    .isEqualToByComparingTo(OPENING.add(OPENING));

assertThat(transactionRepository.count())
    .as("each transfer writes one debit leg and one credit leg")
    .isEqualTo(20 * 2L);
```

Deliberately not @Transactional: a transactional test would roll everything into one transaction and the threads would never see each other.

11.2 Bugs these tests actually found

Bug	How it surfaced	Fix
Concurrent transfers returned HTTP 500	Twelve parallel transfers against the running API: 3 applied, 9 failed with 500.	Bounded retry with jittered backoff in a new bean outside the transaction boundary, plus mapping lock failures to a retryable 409.
Unknown endpoints returned 500	SecurityAndApiIT expected 404 for a typo'd path.	Handle NoResourceFoundException as well as NoHandlerFoundException.
Scheduled interest ran without a transaction	Found by review: a @Transactional method called on 'this' bypasses the proxy.	Moved the scheduler into its own bean so the call crosses a proxy.
Seeded history collapsed onto one day	Charts were flat: JPA auditing stamps @CreateDate unconditionally on insert.	Backdate the seeded rows with a direct UPDATE after the insert, leaving auditing intact for real writes.

What to say about this in an interview

Being able to name a real bug you found, explain the mechanism, and describe the fix is far more convincing than a project where nothing ever went wrong. The concurrency 500 is the best story here: locking was already correct - no money was ever lost - but correctness alone was not an acceptable user experience, and fixing it required understanding why the retry could not live inside the service method.

CHAPTER 12

Deployment

Docker, Compose, and configuration

12

12.1 Running it

```
# Everything, including PostgreSQL:
docker compose up --build

Web app      http://localhost:3000
Swagger UI   http://localhost:3000/swagger-ui.html
API          http://localhost:8080/api/v1

# Or locally, without Docker (uses an in-memory H2 database):
cd backend && ./mvnw spring-boot:run      # needs JDK 21+
cd frontend && npm install && npm run dev  # needs Node 20+ -> localhost:5173
```

12.2 Multi-stage images

Both Dockerfiles build in one stage and ship from another. The backend compiles with a full JDK and Maven, then copies only the jar into a JRE-only runtime image - roughly 200 MB instead of 800 MB, with the entire build toolchain absent from the running container. The frontend builds with Node and ships with nginx; the final image contains no Node, no node_modules and no source.

Layer ordering is deliberate: the POM (and package.json) is copied and dependencies resolved before the source is copied. Docker caches per layer, so editing a Java file does not re-download the dependency tree.

The backend container runs as an unprivileged user rather than root, so a compromise of the process does not start with root inside the container.

12.3 Service orchestration

docker-compose.yml defines three services. The dependency conditions matter: the API waits for the database's healthcheck rather than merely for the container to start, because Flyway runs at boot and would fail against a PostgreSQL still initialising. The web tier likewise waits for the API to report healthy.

```
backend:
  depends_on:
    db:
      condition: service_healthy      # not just "started"
  environment:
    SPRING_PROFILES_ACTIVE: postgres
    DB_URL: jdbc:postgresql://db:5432/novabank
    JWT_SECRET: ${JWT_SECRET:-...}    # overridden in any real deployment
```

12.4 Configuration and secrets

Every tunable - JWT secret and lifetimes, CORS origins, database URL and credentials, transaction limits - is bound to a typed AppProperties record and overridable by an environment variable. The same jar therefore runs in development, Docker and production without a rebuild.

The secret in `application.yml` is a development default and is labelled as such. The application refuses to start if the configured secret is shorter than 32 characters, because HS256 requires at least 256 bits of key material and a short key would sign weakly rather than fail loudly.

12.5 Production gaps

This is a demo deployment, and it is worth being able to list what a real one would add:

- TLS termination, HSTS, and a real certificate.
- Secrets from a manager (Vault, AWS Secrets Manager) rather than environment variables.
- Rate limiting on the auth endpoints, and account lockout after repeated failures.
- Centralised structured logging with a correlation id per request.
- Metrics scraped from Actuator into Prometheus, with alerting.
- Database backups, point-in-time recovery, and a tested restore.
- A CI pipeline running the test suite and a dependency vulnerability scan on every push.

CHAPTER 13

Limitations and Roadmap

An honest account of what is not built

13

Knowing the boundaries of your own system is a stronger signal than claiming it has none. These are the gaps, stated plainly.

13.1 Known limitations

Limitation	Consequence	What a real system does
Tokens in localStorage	Readable by any JavaScript on the page, so XSS can steal a session.	httpOnly, Secure, SameSite cookies plus CSRF tokens.
No rate limiting	Login is brute-forceable given enough time.	Per-IP and per-account throttling, exponential lockout, CAPTCHA.
Single currency	All amounts are INR; there is no FX.	Currency per account, an FX rate service, and settlement rules.
No external transfers	Money only moves between NovaBank accounts.	NEFT/RTPS/UPI rails, with asynchronous settlement and reversal handling.
Simplified interest	Simple monthly accrual, not daily compounding on minimum balance.	Daily balance accrual with the product's exact compounding rules.
No KYC workflow	The KYC flag is a boolean an admin toggles.	Document upload, verification against a registry, an approval queue.
No two-factor authentication	A stolen password is sufficient to sign in.	TOTP or SMS OTP, and step-up authentication for high-value transfers.
No soft delete on payees	A removed payee is gone.	Soft delete, so historical transfers still resolve the payee name.
JavaScript, not TypeScript	Type errors surface at runtime.	TypeScript with generated API types.

13.2 Roadmap, in priority order

1. Migrate the frontend to TypeScript, with request/response types generated from the OpenAPI document so the client and server cannot disagree.
2. Move tokens to httpOnly cookies and reintroduce CSRF protection.
3. Add rate limiting (Bucket4j) and account lockout on the auth endpoints.
4. Replace H2 in the integration tests with Testcontainers, so tests run against the same PostgreSQL version as production and the lock semantics under test are the real ones.
5. Add a scheduled transfers feature, reusing the existing idempotency machinery.
6. Publish domain events (TransferCompleted) to support notifications and, later, a read model.
7. Add a CI pipeline: build, test, dependency scan, and image publish on every push.
8. Add structured logging with a correlation id, and Prometheus metrics from Actuator.

CHAPTER 14

Demo Script

How to present this in an interview

14

A ten-minute run-through that shows the interesting parts rather than clicking through every screen. Start the stack before the call, not during it.

14.1 Before you start

- docker compose up --build, or run the two local commands - and confirm both are up.
- Have three tabs ready: the app, Swagger UI, and your editor on TransactionService.java.
- Have ./mvnw test finished, so you can show 27 passing tests without waiting.

14.2 The run-through

Time	Beat	What to say and show
0:00	Frame it in one sentence	"A full-stack banking platform - Spring Boot and React - where the interesting part is that money movement is atomic, concurrency-safe and idempotent, not that it does CRUD."
0:30	Sign in as a customer	Use the one-click demo login. Land on the dashboard; point out that it is a single aggregated endpoint rather than five parallel calls.
1:30	Make a transfer	Pick a saved payee, send an amount, show the receipt with its reference. Mention that the client minted an idempotency key for this attempt.
2:30	Show the double entry	Open Transactions on both sides, or the CSV export. Two rows, one reference, one debit and one credit.
3:30	Switch to the admin	Sign in as admin. Show holdings and the audit trail - including a failed attempt, which is the point of auditing.
4:30	Freeze an account	Freeze it, then try to debit it as the customer and show the 422 with a specific code. Then note credits are still accepted, and why that is the right behaviour.
5:30	Open the code: the lock ordering	Show the min/max lock acquisition in transfer() and explain the A->B / B->A deadlock it prevents. This is the strongest ninety seconds of the demo.
7:00	Show the concurrency test	TransferConcurrencyIT: 20 threads on a CountdownLatch, asserting the total is conserved and exactly 40 ledger rows were written.
8:00	Tell the bug story	Twelve concurrent transfers returned 500s. Explain lock-contention aborts, why the retry had to move outside the transaction boundary, and the before/after numbers.
9:00	Close with the limitations	Name two or three from Chapter 13 unprompted - localStorage tokens, no rate limiting, no external rails. Ending on what you would do next reads as engineering maturity.

14.3 Three things to have ready

If they ask...	Go straight to
"How do you handle concurrency?"	The lost-update table in 7.1, then the lock ordering in 7.4. Draw the deadlock if there is a whiteboard.
"What was the hardest bug?"	The concurrency 500s: the symptom, the mechanism, why the retry could not live inside the service, and the before/after measurement.
"What would you do differently?"	TypeScript, httpOnly cookies, and Testcontainers instead of H2 - and say why each, briefly.

NovaBank Bank Management System · Gurudutta Pradhan
guruduttapradhan140@gmail.com